

ARTI 404 – Image Processing

Lab 4: Intensity Transformations and Filtering – Spatial Domain

Student Name: Abdullah Hejji ALnuwaysir

Student ID: 2240005501

Completed Lab Code

ARTI 404 – Image Processing

Lab 4: Intensity Transformations and Filtering – Spatial Domain

Student Name: Abdullah Hejji alnuwaysir
Student ID: 2240005501

This notebook uses only the original input images stored in the 'images' folder. All processed images, thresholding results, histograms, contrast stretching, histogram equalization, and histogram matching results are generated by the code when the notebook is run.

The implementation follows the Student Procedural Manual and the listed scikit-image resources.

```
# Run once in Jupyter if needed: %pip install scikit-image

import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import io, exposure
from skimage.exposure import match_histograms

# Input images are kept here. These are original images only.
IMAGE_DIR = 'images'
OUTPUT_DIR = 'outputs'
os.makedirs(OUTPUT_DIR, exist_ok=True)

threshold_path = os.path.join(IMAGE_DIR, 'threshold_input.png')
moon_path = os.path.join(IMAGE_DIR, 'moon.png')
rocket_path = os.path.join(IMAGE_DIR, 'rocket.png')
chelsea_path = os.path.join(IMAGE_DIR, 'chelsea.png')

print('Input images:')
for path in [threshold_path, moon_path, rocket_path, chelsea_path]:
    print('-', path)
```

Task 1: Thresholding

```
# Load an ORIGINAL grayscale image from the images folder
img = cv2.imread(threshold_path, cv2.IMREAD_GRAYSCALE)

if img is None:
    raise FileNotFoundError(f'Could not load image: {threshold_path}')

threshold_values = [0, 50, 100, 150, 200]

fig, axes = plt.subplots(1, 6, figsize=(16, 3))
axes[0].imshow(img, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('Original')
axes[0].axis('off')

for ax, t in zip(axes[1:], threshold_values):
    _, binary = cv2.threshold(img, t, 255, cv2.THRESH_BINARY)
    ax.imshow(binary, cmap='gray', vmin=0, vmax=255)
    ax.set_title(f'T = {t}')
    ax.axis('off')

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'task1_thresholding.png'), dpi=150, bbox_inches='tight')
plt.show()
```

Task 2: Histogram Processing – Contrast Stretching (2nd–98th Percentiles)

```
# Load the ORIGINAL moon image from the images folder
moon = io.imread(moon_path)

if moon.ndim != 2:
    moon = cv2.cvtColor(moon, cv2.COLOR_RGB2GRAY)
```

```

p2, p98 = np.percentile(moon, (2, 98))
moon_rescale = exposure.rescale_intensity(moon, in_range=(p2, p98))

fig, axes = plt.subplots(2, 2, figsize=(12, 9))

axes[0, 0].imshow(moon, cmap='gray')
axes[0, 0].axis('off')
axes[0, 0].set_title('Original Moon')

axes[0, 1].hist(moon.ravel(), bins=256, range=(0, 255))
axes[0, 1].set_title('Original Histogram')
axes[0, 1].set_xlabel('Gray level values')
axes[0, 1].set_ylabel('Number of pixels')

axes[1, 0].imshow(moon_rescale, cmap='gray')
axes[1, 0].axis('off')
axes[1, 0].set_title('Contrast Stretch: 2nd-98th Percentiles')

axes[1, 1].hist(moon_rescale.ravel(), bins=256, range=(0, 255))
axes[1, 1].set_title('Histogram After Contrast Stretching')
axes[1, 1].set_xlabel('Gray level values')
axes[1, 1].set_ylabel('Number of pixels')

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'contrast_stretch_2_98.png'), dpi=150, bbox_inches='tight')
plt.show()

```

Assessment Task 1: Contrast Stretching (3rd–80th Percentiles)

```

# Rescale the ORIGINAL moon image using the 3rd and 80th percentiles
p3, p80 = np.percentile(moon, (3, 80))
moon_task1 = exposure.rescale_intensity(moon, in_range=(p3, p80))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].imshow(moon_task1, cmap='gray')
axes[0].axis('off')
axes[0].set_title('Moon: 3rd-80th Percentile Stretch')

axes[1].hist(moon_task1.ravel(), bins=256, range=(0, 255))
axes[1].set_xlabel('Gray level values')
axes[1].set_ylabel('Number of pixels')
axes[1].set_title('Histogram: 3rd-80th Percentile Stretch')

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'assessment_task1_3rd_80th.png'), dpi=150, bbox_inches='tight')
plt.show()

```

Assessment Task 2: Histogram Equalization

```

# Histogram equalization using the scikit-image exposure.equalize_hist function
moon_equalized = exposure.equalize_hist(moon)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].imshow(moon_equalized, cmap='gray')
axes[0].axis('off')
axes[0].set_title('Moon: Histogram Equalization')

axes[1].hist(moon_equalized.ravel(), bins=256, range=(0, 1))
axes[1].set_xlabel('Normalized intensity')
axes[1].set_ylabel('Number of pixels')
axes[1].set_title('Histogram After Equalization')

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'assessment_task2_equalization.png'), dpi=150, bbox_inches='tight')
plt.show()

```

Assessment Task 3: Histogram Matching

```

# Load ORIGINAL source and reference images from the images folder
source = io.imread(chelsea_path) # Chelsea = source image
reference = io.imread(rocket_path) # Rocket = reference image

# Match the source image histogram to the reference image histogram

```

```

matched = match_histograms(source, reference, channel_axis=-1)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(source)
axes[0].set_title('Source: Chelsea')
axes[0].axis('off')

axes[1].imshow(reference)
axes[1].set_title('Reference: Rocket')
axes[1].axis('off')

axes[2].imshow(matched)
axes[2].set_title('Histogram Matched')
axes[2].axis('off')

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'assessment_task3_histogram_matching.png'), dpi=150, bbox_inches='tight')
plt.show()

# Compare RGB histograms before and after histogram matching
images = [source, reference, matched]
titles = ['Chelsea (Source)', 'Rocket (Reference)', 'Matched']
channels = ['Red', 'Green', 'Blue']

fig, axes = plt.subplots(3, 3, figsize=(14, 10))

for col, (image, title) in enumerate(zip(images, titles)):
    for row, channel in enumerate(channels):
        hist, bins = exposure.histogram(image[..., row], source_range='dtype')
        axes[row, col].plot(bins, hist)
        axes[row, col].set_title(f'{title} - {channel}')
        axes[row, col].set_xlabel('Intensity')
        axes[row, col].set_ylabel('Pixel count')

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'assessment_task3_rgb_histograms.png'), dpi=150, bbox_inches='tight')
plt.show()

```

Generated Results

The following files are created in the `outputs` folder only after the notebook is run.

```

print('Generated result files:')
for filename in sorted(os.listdir(OUTPUT_DIR)):
    print('-', filename)

```